

Compte rendu — Tests unitaires Jest & Interface Handlebars

Maganga, Emilio · 08 September 2026 · Projet emilio-express-courses

N/A

N/A

1. Description des tests unitaires implémentés

La suite de tests couvre l'ensemble des couches applicatives : les services métier (accès aux données), les schémas de validation Joi, le middleware d'erreur centralisé, les helpers Handlebars, ainsi que l'API REST et l'interface web de bout en bout via Supertest. Chaque fichier de test utilise une base SQLite en mémoire (DB_FILE= ':memory: '), réinitialisée avant chaque cas de test, garantissant des tests isolés et reproductibles sans dépendre ni modifier la base de données de développement.

Fichier de test	Portée
tests/student-service.test.mjs	CRUD complet du service étudiants
tests/course-service.test.mjs	CRUD complet du service cours
tests/subscription-service.test.mjs	Inscriptions, contraintes d'intégrité, jointures
tests/error-and-validation.test.mjs	Schémas Joi et middleware d'erreur centralisé
tests/helpers.test.mjs	Helpers Handlebars personnalisés
tests/api.test.mjs	API REST JSON (Partie III) via Supertest
tests/web.test.mjs	Interface Handlebars (Partie IV) via Supertest

2. Couverture de code et résultats

Résultat global obtenu avec `npm test (Jest + --coverage)` :

Fichier	Statements	Branches	Functions	Lines
Total	81.51%	73.55%	87.27%	81.93%
middlewares/error-handler.mjs	100%	77.77%	100%	100%
middlewares/validation.mjs	100%	100%	100%	100%
routes/web-routes.mjs	77.38%	58.82%	81.25%	78.18%
services/course-service.mjs	89.47%	95%	100%	89.47%
services/student-service.mjs	88.88%	93.75%	100%	88.88%
services/subscription-service.mjs	77.77%	100%	88.88%	77.27%

3. Captures d'écran du rapport de couverture

Rapport HTML généré par Jest/Istanbul (coverage/lcov-report/index.html) :

All files

81.51% Statements

247/303

73.55% Branches

89/121

87.27% Functions

48/55

81.93% Lines

245/299

Press *n* or *j* to go to the next uncovered block, *b*, *p* or *k* for the previous block.

Filter:

File ^		Statements ^		Branches ^		Functions ^		Lines ^	
middlewares		100%		16/16		9/11		100%	16/16
routes		77.38%		130/168		40/68		78.18%	129/165
services		84.87%		101/119		40/42		84.74%	100/118

Code coverage generated by [Istanbul](#) at 2026-09-08T14:49:40.812Z

4. Résumé des cas de test pour chaque service

StudentService

- Création d'un étudiant et retour de son id auto-incrémenté
- Rejet d'un doublon de matricule (contrainte UNIQUE)
- Recherche par id numérique et par matricule
- Retour de null quand aucun étudiant ne correspond
- Mise à jour partielle et cas d'étudiant introuvable
- Suppression par id ou par matricule, avec retour booléen
- Récupération de la liste complète

CourseService

- Création avec crédits par défaut (0) et avec crédits explicites
- Rejet d'un doublon de code de cours
- Recherche par id ou par code
- Mise à jour des crédits et cas de cours introuvable
- Suppression et récupération de la liste complète

SubscriptionService

- Inscription valide (étudiant + cours existants)
- Rejet si l'étudiant ou le cours référencé n'existe pas
- Rejet d'une double inscription au même cours (contrainte UNIQUE composite)
- Récupération croisée : inscriptions par étudiant, par cours
- Récupération croisée : étudiants d'un cours, cours d'un étudiant (jointures SQL)
- Suppression d'une inscription et cas d'inscription introuvable
- Liste complète avec étudiant et cours joints (getAll())

Validation (Joi) & gestion d'erreurs

- Acceptation des payloads valides pour étudiants, cours, inscriptions (création et mise à jour partielle)
- Rejet des champs obligatoires manquants et des types invalides
- Middleware d'erreur centralisé : statut par défaut 500, respect du statut porté par l'erreur

API REST (Partie III) & interface web (Partie IV)

- Cycle CRUD complet testé via HTTP réel (Supertest) pour étudiants, cours et inscriptions
- Codes de statut vérifiés : 200, 201, 302 (redirections web), 400, 404
- Rendu HTML des pages principales, formulaires, recherche, pagination des étudiants
- Réaffichage des formulaires avec messages d'erreur en cas de validation échouée

5. Configuration de Handlebars

Le moteur de vues est configuré dans `server.mjs` via `express-handlebars`, avec un layout par défaut et des helpers importés depuis un module dédié (testable indépendamment du serveur) :

```
app.engine('handlebars', engine({
  defaultLayout: 'main',
  extname: '.handlebars',
  helpers: handlebarsHelpers // importé de ./helpers/handlebars-helpers.mjs
}));
app.set('view engine', 'handlebars');
app.set('views', path.join(__dirname, 'views'));
```

6. Création des templates pour étudiants, cours et inscriptions

Entité	Vues créées
Étudiants	views/students/list.handlebars, detail.handlebars, form.handlebars
Cours	views/courses/list.handlebars, detail.handlebars, form.handlebars
Inscriptions	views/subscriptions/list.handlebars, form.handlebars
Tableau de bord	views/home.handlebars
Layout	views/layouts/main.handlebars

7. Utilisation de helpers et partials Handlebars

Helpers personnalisés

Helper	Usage
formatDate	Formate une date (Moment.js) — utilisé pour les dates d'inscription et le pied de page
eq	Comparaison stricte, utilisée pour présélectionner une option de formulaire
gt	Comparaison numérique >, disponible pour la logique conditionnelle des vues
sum	Addition de deux valeurs numériques
json	Sérialisation JSON pour du débogage côté vue
includes	Test d'appartenance à un tableau
slice	Découpage de chaîne

Partials réutilisables

- navbar / footer — inclus dans le layout principal
- flash-messages — bannières de succès/erreur alimentées par connect-flash
- error-message — liste d'erreurs de validation dans les formulaires
- student-card / course-card — cartes réutilisées dans le tableau de bord et les fiches détail croisées

- `form-input` / `form-select` — champs de formulaire génériques (étudiants, cours, inscriptions)

8. Améliorations possibles

- **Pagination en base de données** : la pagination est actuellement réalisée en mémoire (`Array.slice`) après un `SELECT *` ; passer à des requêtes `LIMIT/OFFSET` serait plus performant sur un grand volume de données.
- **Couverture de code des routes web** : la couverture des routes Handlebars (N/A%) pourrait être étendue avec des cas d'erreur supplémentaires (ex : suppression d'une ressource déjà supprimée en simultané).
- **Tests end-to-end navigateur** : ajouter des tests Puppeteer/Playwright pilotant un vrai navigateur pour valider le rendu visuel et les interactions JS (auto-masquage des messages flash, confirmations de suppression).
- **Authentification** : l'interface d'administration n'est protégée par aucun mécanisme d'authentification ; ajouter une session utilisateur avant toute mise en production.
- **Internationalisation** : externaliser les textes actuellement codés en dur en français pour permettre une éventuelle traduction.

Conclusion

La mise en place d'une suite de tests Jest complète, combinée à l'interface Handlebars consommant les mêmes services que l'API REST, a permis de valider le comportement de l'application à la fois au niveau unitaire et au niveau des parcours utilisateur complets (HTTP réel via Supertest). L'isolation des tests via une base SQLite en mémoire s'est révélée être l'un des choix techniques les plus importants du projet : elle garantit des résultats déterministes sans jamais toucher aux données de développement.

Annexe — Lien du repository GitHub : <https://github.com/tooluxarala/emilio-express-courses>